

# 12 · PL/SQL Fundamentals

**Why this matters in interviews.** For Oracle roles PL/SQL is a whole interview round: *block structure, variables and %TYPE, IF / CASE / loops, anonymous vs stored code, DBMS\_OUTPUT*. This chapter is the foundation; Chapters 13–15 build procedures, functions, packages, cursors and triggers on it.

**In this chapter:** what PL/SQL is · the block · lexical elements (comments, delimiters, literals) · data types and subtypes · `NULL` · variables, constants and scope · operators · `IF`, `CASE` · loops · strings · arrays ( `VARRAY` ).

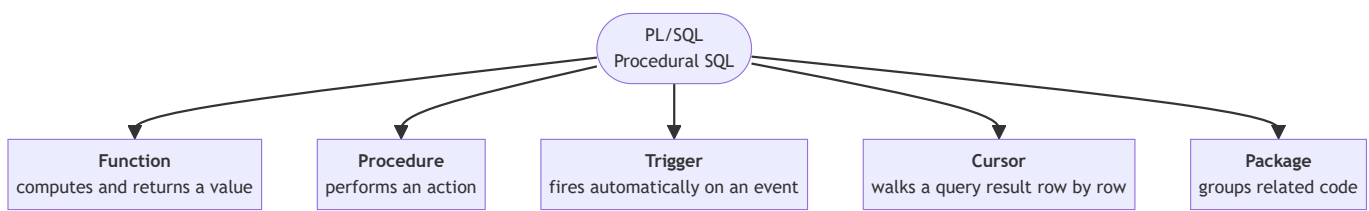
**Convention.** Outputs shown under a program are what `DBMS_OUTPUT` prints when you run it in SQL Developer with `SET SERVEROUTPUT ON;`. Every program is finished with a `/` on its own line.

## 12.1 What is PL/SQL?

**PL/SQL** (Procedural Language extensions to **SQL**) is Oracle's procedural programming language. It lets you write **variables, conditions, loops, error handling and reusable code** around your SQL statements.

|              | SQL                                                              | PL/SQL                                                  |
|--------------|------------------------------------------------------------------|---------------------------------------------------------|
| Style        | <b>declarative</b> — <i>what</i> you want ("give me these rows") | <b>procedural</b> — <i>how</i> , step by step           |
| Unit         | a single statement                                               | a <b>block</b> of statements                            |
| Control flow | none                                                             | <code>IF</code> , <code>CASE</code> , loops, exceptions |
| Variables    | none                                                             | yes                                                     |
| Runs         | one statement per call                                           | many statements in <b>one</b> call                      |

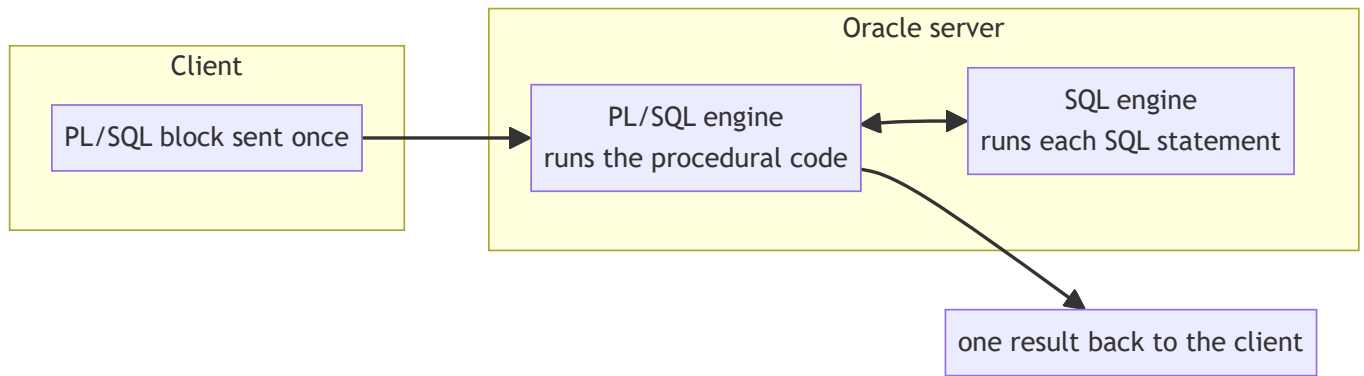
PL/SQL is the language of the database's own program units:



The formal list of **PL/SQL program units**: *PL/SQL block · Function · Package · Package body · Procedure · Trigger · Type · Type body*.

### Why use it?

- **Fewer round trips.** A block is sent to the server once and runs there, instead of many separate SQL calls over the network.
- **Reusable, stored logic** — write a rule once ( `bonus()` ), use it everywhere, keep it secure in the database.
- **Tight SQL integration** — SQL statements sit directly inside the code.
- **Portable across Oracle platforms**, with robust **exception handling**.



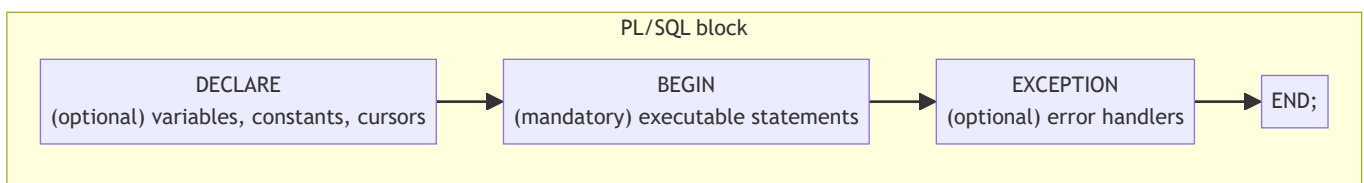
## 12.2 The block

Every PL/SQL program is made of **blocks**. A block has up to **four sections**:

```

DECLARE
    -- 1. declarations: variables, constants, cursors, types, local subprograms (optional)
BEGIN
    -- 2. executable statements (MANDATORY)
EXCEPTION
    -- 3. error handlers (optional)
END;          -- 4. every block ends with END;
/
  
```

| Section            | Keyword   | Required?        | Contains                                                      |
|--------------------|-----------|------------------|---------------------------------------------------------------|
| Declaration        | DECLARE   | optional         | variables, constants, cursors, user types, nested subprograms |
| Executable         | BEGIN ... | <b>mandatory</b> | the SQL and procedural statements that do the work            |
| Exception handling | EXCEPTION | optional         | code that runs when a run-time error occurs                   |
| End                | END;      | mandatory        | terminates the block                                          |



### Hello, World:

```

SET SERVEROUTPUT ON;
DECLARE
    message VARCHAR2(20) := 'Hello, World!';
BEGIN
    DBMS_OUTPUT.PUT_LINE(message);
END;
/
  
```

```
Hello, World!
```

```
PL/SQL procedure successfully completed.
```

- `DBMS_OUTPUT.PUT_LINE` writes a line to a buffer that SQL Developer shows in **Script Output** — only after `SET SERVEROUTPUT ON`.
- The smallest legal block: `BEGIN NULL; END; — NULL;` is the "do nothing" statement.
- The `/` on the last line tells SQL Developer/SQL\*Plus to **run** the block; it is a client signal, not PL/SQL.

## Anonymous vs named blocks

**Anonymous block** — unnamed, **not stored**; compiled and run each time you submit it. `DECLARE` is needed only if you have local variables; with none you can start straight with `BEGIN ... END;`.

```
DECLARE
  x NUMBER;
BEGIN
  x := 10;
  DBMS_OUTPUT.PUT_LINE(x);
END;
/
```

**Named block** (procedure or function) — has a name, is **stored in the database** and compiled once. It has **no DECLARE keyword**: the `IS / AS` section takes the place of `DECLARE` for local variables.

```
CREATE OR REPLACE PROCEDURE my_proc IS
  x NUMBER;           -- local variable, declared after IS
BEGIN
  x := 10;
  DBMS_OUTPUT.PUT_LINE(x);
END;
/
```

|                         | Anonymous (local / unnamed)                                                        | Named (stored)                                             |
|-------------------------|------------------------------------------------------------------------------------|------------------------------------------------------------|
| Name                    | none                                                                               | yes                                                        |
| Stored in the database  | ✗ no                                                                               | ✓ yes                                                      |
| Initialisation and call | done <b>together, in the same program</b> — it executes within the current session | can be <b>created once and called later, from anywhere</b> |
| Compiled                | every run                                                                          | once, when created                                         |
| Reusable / parameters   | no                                                                                 | yes                                                        |
| Typical use             | scripts, tests, one-off tasks                                                      | application logic                                          |

**Local (nested) subprograms.** A procedure or function can also be **declared inside another block's declaration section**. It exists only while that block runs and must be **defined and called in the same program** — the "local procedure" of the lecture, as opposed to a **stored** procedure that lives in the schema. (Chapter 13.)

## Nested blocks

A block can contain other blocks — commonly to give a small piece of code its own `EXCEPTION` section:

```
BEGIN
  DBMS_OUTPUT.PUT_LINE('outer starts');
  BEGIN                                -- inner block
    DBMS_OUTPUT.PUT_LINE('inner runs');
  END;
  DBMS_OUTPUT.PUT_LINE('outer ends');
END;
/
```

## 12.3 Lexical elements

### Comments

Comments are explanatory statements that help readers of the source code; **the compiler ignores them**. Single-line comments start with `--` (double hyphen); multi-line comments are enclosed by `/*` and `*/`.

```
DECLARE
  -- variable declaration
  message VARCHAR2(20) := 'Hello, World!';
BEGIN
  /*
   * PL/SQL executable statement(s)
   */
  DBMS_OUTPUT.PUT_LINE(message);
END;
/
```

Hello, World!

PL/SQL procedure successfully completed.

### Delimiters

| Delimiter            | Meaning                                                    |
|----------------------|------------------------------------------------------------|
| <code>+ - * /</code> | addition, subtraction / negation, multiplication, division |

| Delimiter                    | Meaning                                                                |
|------------------------------|------------------------------------------------------------------------|
| <b>**</b>                    | exponentiation                                                         |
| <b>%</b>                     | attribute indicator ( <b>%TYPE</b> , <b>%ROWTYPE</b> , <b>%FOUND</b> ) |
| <b>'</b>                     | character-string delimiter                                             |
| <b>"</b>                     | quoted-identifier delimiter                                            |
| <b>.</b>                     | component selector ( <b>rec.name</b> , <b>pkg.proc</b> )               |
| <b>( , )</b>                 | expression or list delimiter                                           |
| <b>,</b>                     | item separator                                                         |
| <b>:</b>                     | host (bind) variable indicator ( <b>:new</b> , <b>:x</b> )             |
| <b>=</b>                     | relational operator ( <b>equality</b> — assignment is <b>:=</b> )      |
| <b>@</b>                     | remote access indicator (database link)                                |
| <b>;</b>                     | statement terminator                                                   |
| <b>:=</b>                    | <b>assignment</b> operator                                             |
| <b>=&gt;</b>                 | association operator (named parameters)                                |
| <b>\\ </b>                   | concatenation operator                                                 |
| <b>&lt;&lt; &gt;&gt;</b>     | label delimiters (begin and end)                                       |
| <b>/* */</b>                 | multi-line comment delimiters                                          |
| <b>--</b>                    | single-line comment indicator                                          |
| <b>..</b>                    | range operator ( <b>1..10</b> )                                        |
| <b>&lt; &gt; &lt;= &gt;=</b> | relational operators                                                   |
| <b>&lt;&gt; != ~= ^=</b>     | different spellings of <b>not equal</b>                                |

## Literals

A **literal** is an explicit numeric, character, string or Boolean value not represented by an identifier — e.g. **TRUE** , **786** , **NULL** , **'tutorialspoint'** . **PL/SQL literals are case-sensitive** ( **'Abc'**  $\neq$  **'abc'** ), while keywords and identifiers are not.

| Kind                 | Examples                                                                                                                                                                                                            |
|----------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Numeric</b>       | <b>050</b> <b>78</b> <b>-14</b> <b>0</b> <b>+32767</b> <b>.</b> <b>6.6667</b> <b>0.0</b> <b>-12.0</b> <b>3.14159</b> <b>+7800.00</b> <b>.</b> <b>6E5</b> <b>1.0E-8</b> <b>3.14159e0</b> <b>-1E38</b> <b>-9.5e-3</b> |
| <b>Character</b>     | <b>'A'</b> <b>'%'</b> <b>'9'</b> <b>' '</b> <b>'z'</b> <b>'('</b>                                                                                                                                                   |
| <b>String</b>        | <b>'Hello, world!'</b> <b>'Tutorials Point'</b> <b>'19-NOV-12'</b>                                                                                                                                                  |
| <b>Boolean</b>       | <b>TRUE</b> <b>FALSE</b> <b>NULL</b>                                                                                                                                                                                |
| <b>Date and time</b> | <b>DATE</b> <b>'1978-12-25'</b> <b>TIMESTAMP</b> <b>'2012-10-29 12:01:01'</b>                                                                                                                                       |

Identifiers are **case-insensitive** ( **Total** = **TOTAL** ), up to 30 characters, start with a letter, and cannot be reserved words ( **BEGIN** , **END** , **IF** ...).

## 12.4 Data types

A variable's type decides what values it can hold and how much memory it needs. PL/SQL's **scalar** types come in four categories:

| Category         | Values                       | Use        |
|------------------|------------------------------|------------|
| <b>Numeric</b>   | numbers                      | arithmetic |
| <b>Character</b> | single characters or strings | text       |
| <b>Boolean</b>   | TRUE , FALSE , NULL          | logic      |
| <b>Datetime</b>  | dates and times              | timestamps |

**PL/SQL vs SQL.** The two share most type names but not limits: a PL/SQL `VARCHAR2` variable holds up to **32,767 bytes**, but a `VARCHAR2` *table column* holds 4,000. And `BOOLEAN` exists **only in PL/SQL** (not as an SQL column type before 23ai).

### Numeric types

| Type                                                            | Description                                                                                         |
|-----------------------------------------------------------------|-----------------------------------------------------------------------------------------------------|
| <code>PLS_INTEGER</code>                                        | signed integer, -2,147,483,648 to 2,147,483,647, <b>32 bits</b> (fast)                              |
| <code>BINARY_INTEGER</code>                                     | same range, 32 bits                                                                                 |
| <code>BINARY_FLOAT</code>                                       | single-precision IEEE 754 floating point                                                            |
| <code>BINARY_DOUBLE</code>                                      | double-precision IEEE 754 floating point                                                            |
| <code>NUMBER(prec, scale)</code>                                | fixed- or floating-point number, absolute value 1E-130 up to (not including) 1.0E126; can also be 0 |
| <code>DEC(prec, scale)</code>                                   | ANSI fixed-point, maximum precision <b>38</b> digits                                                |
| <code>DECIMAL(prec, scale)</code>                               | IBM fixed-point, maximum 38 digits                                                                  |
| <code>NUMERIC(prec, scale)</code>                               | floating type, maximum 38 digits                                                                    |
| <code>DOUBLE PRECISION</code>                                   | ANSI floating point, 126 binary digits ( $\approx$ 38 decimal)                                      |
| <code>FLOAT</code>                                              | ANSI and IBM floating point, 126 binary digits ( $\approx$ 38 decimal)                              |
| <code>INT</code> / <code>INTEGER</code> / <code>SMALLINT</code> | ANSI (and IBM) integer, maximum 38 decimal digits                                                   |
| <code>REAL</code>                                               | floating point, 63 binary digits ( $\approx$ 18 decimal)                                            |

`PLS_INTEGER` is the fastest for loop counters and counters in general (it uses machine arithmetic); `NUMBER` is the general-purpose type and the one used for table columns.

### Character types

| Type              | Description                                                               |
|-------------------|---------------------------------------------------------------------------|
| <code>CHAR</code> | <b>fixed-length</b> character string, maximum 32,767 bytes (blank-padded) |

| Type      | Description                                                                                 |
|-----------|---------------------------------------------------------------------------------------------|
| VARCHAR2  | <b>variable-length</b> character string, maximum 32,767 bytes                               |
| RAW       | variable-length binary/byte string, maximum 32,767 bytes — <b>not interpreted</b> by PL/SQL |
| NCHAR     | fixed-length <b>national</b> (Unicode) string, maximum 32,767 bytes                         |
| NVARCHAR2 | variable-length national string, maximum 32,767 bytes                                       |
| LONG      | variable-length character string, maximum 32,760 bytes (legacy)                             |

## Datetime types

DATE, TIMESTAMP, TIMESTAMP WITH TIME ZONE, TIMESTAMP WITH LOCAL TIME ZONE, and the INTERVAL types. Valid field values (Chapter 01): YEAR -4712...9999 (no year 0) · MONTH 01–12 · DAY 01–31 · HOUR 00–23 · MINUTE 00–59 · SECOND 00–59.9(n).

## Large object (LOB) types

LOBs hold large items — **text, graphic images, video clips and sound waveforms** — and allow efficient, random, **piecewise access** to them.

| Type  | Used to store                                                              | Size                     |
|-------|----------------------------------------------------------------------------|--------------------------|
| BFILE | large binary objects in <b>operating-system files outside the database</b> | system-dependent; ≤ 4 GB |
| BLOB  | large binary objects <b>in the database</b>                                | 8 – 128 TB               |
| CLOB  | large blocks of character data in the database                             | 8 – 128 TB               |
| NCLOB | large blocks of NCHAR (Unicode) data in the database                       | 8 – 128 TB               |

## User-defined subtypes

A **subtype** is a **subset of another data type — its base type**. It has the same valid operations as the base type but only a subset of its valid values. PL/SQL predefines several subtypes in package STANDARD, for instance:

```
SUBTYPE CHARACTER IS CHAR;
SUBTYPE INTEGER IS NUMBER(38,0);
```

You can define your own to give meaning to a type and centralise its size:

```
DECLARE
    SUBTYPE name IS CHAR(20);
    SUBTYPE message IS VARCHAR2(100);
    salutation name;
    greetings message;
BEGIN
    salutation := 'Reader ';
    greetings := 'Welcome to the World of PL/SQL';
    DBMS_OUTPUT.PUT_LINE('Hello ' || salutation || greetings);
END;
/
```

```
Hello Reader                Welcome to the World of PL/SQL
```

```
PL/SQL procedure successfully completed.
```

(`salutation` is `CHAR(20)`, so `'Reader '` (7 characters) is blank-padded to 20 — that fixed-length padding is why "Reader" is followed by a wide gap before "Welcome". Declare it `VARCHAR2(20)` and the gap disappears.)

## 12.5 NULL in PL/SQL

PL/SQL `NULL` represents **missing or unknown data**. It is **not** an integer, a character or any specific type. Note that `NULL` is **not the same as an empty string or the null character** `'\0'` — although, in Oracle, an *empty string* assigned to a `VARCHAR2` is treated as `NULL`. A null can be **assigned**, but it **cannot be equated with anything, including itself**.

```
DECLARE
    a NUMBER;                -- no initial value → NULL
BEGIN
    IF a = NULL THEN         -- never TRUE, not even for NULL = NULL
        DBMS_OUTPUT.PUT_LINE('equal');
    ELSIF a IS NULL THEN     -- the correct test
        DBMS_OUTPUT.PUT_LINE('a is null');
    END IF;
END;
/
```

```
a is null
```

**Trap.** Every comparison with `NULL` is **UNKNOWN**, and an `IF` treats `UNKNOWN` as **not TRUE** — so the `ELSE` branch runs. `IF salary > 5000 THEN ... ELSE ... END IF;` with a `NULL` salary goes to `ELSE`. Always test with `IS NULL` / `IS NOT NULL`, or wrap with `NVL`.

## 12.6 Variables, constants and scope

### Declaring a variable

```
variable_name [CONSTANT] datatype [NOT NULL] [:= | DEFAULT initial_value];
```

`variable_name` is a valid identifier; `datatype` is a valid PL/SQL type or a user-defined type.

```

sales      NUMBER(10, 2);
pi         CONSTANT DOUBLE PRECISION := 3.1415;
name       VARCHAR2(25);
address    VARCHAR2(100);
counter    BINARY_INTEGER := 0;
greetings  VARCHAR2(20) DEFAULT 'Have a Good Day';

```

- **Initialise** with `:=` or `DEFAULT`. A variable with no initial value starts as **NULL**.
- **NOT NULL** forbids NULL — and then you **must give an initial value**.
- When you give a size, scale or precision limit it is a **constrained declaration**; constrained declarations require **less memory** than unconstrained ones.
- `:=` **assigns**, `=` **compares**. `x := 5;` sets; `IF x = 5 THEN` tests.

## Declaring a constant

A **constant** is declared with the `CONSTANT` keyword. It **requires an initial value** and **does not allow that value to be changed**.

```

DECLARE
    -- constant declaration
    pi CONSTANT NUMBER := 3.141592654;
    -- other declarations
    radius      NUMBER(5,2);
    dia         NUMBER(5,2);
    circumference NUMBER(7,2);
    area        NUMBER(10,2);
BEGIN
    -- processing
    radius      := 9.5;
    dia         := radius * 2;
    circumference := 2.0 * pi * radius;
    area        := pi * radius * radius;
    -- output
    DBMS_OUTPUT.PUT_LINE('Radius: ' || radius);
    DBMS_OUTPUT.PUT_LINE('Diameter: ' || dia);
    DBMS_OUTPUT.PUT_LINE('Circumference: ' || circumference);
    DBMS_OUTPUT.PUT_LINE('Area: ' || area);
END;
/

```

```

Radius: 9.5
Diameter: 19
Circumference: 59.69
Area: 283.53

```

PL/SQL procedure successfully completed.

`circumference` is `NUMBER(7,2)`, so the exact result 59.690260... is **rounded to two decimals** on assignment; likewise `area` (283.5287...). Assigning to `pi` would be a compile error ( `PLS-00363: expression 'PI' cannot be used as an assignment target` ).

### **%TYPE — declare a variable like a column**

`table.column%TYPE` gives a variable **the same data type as a column** — if the column later changes, the variable follows it automatically.

`SELECT ... INTO` **assigns the values of a query to PL/SQL variables:**

```
DECLARE
    c_id    customers.id%TYPE := 1;
    c_name  customers.name%TYPE;
    c_addr  customers.address%TYPE;
    c_sal   customers.salary%TYPE;
BEGIN
    SELECT name, address, salary
    INTO   c_name, c_addr, c_sal
    FROM   customers
    WHERE  id = c_id;

    DBMS_OUTPUT.PUT_LINE('Customer ' || c_name || ' from ' || c_addr || ' earns ' || c_sal);
END;
/
```

Customer Ramesh from Ahmedabad earns 2000

PL/SQL procedure successfully completed.

**SELECT ... INTO must return exactly one row.** Zero rows raises `NO_DATA_FOUND`; more than one raises `TOO_MANY_ROWS`. (For many rows use a cursor — Chapter 14.) Note that here `INTO` means "*assign to variables*" — a different meaning from SQL Server's `SELECT ... INTO newtable`.

**Naming tip.** A variable named like a column ( `salary`, `name` ) can **shadow** the column inside a SQL statement ( `WHERE salary = salary` compares the variable with itself). Prefix variables: `v_salary`, `v_name`, and parameters `p_...`.

## **Scope: local and global variables**

Blocks can be **nested**. A variable declared in an **inner block** is **not accessible to the outer block**; a variable declared in an **outer block** is accessible to **all nested inner blocks**.

- **Local** variables — declared in an inner block, invisible outside it.
- **Global** variables — declared in the **outermost block** or a **package**.

```

DECLARE
    -- Global variables
    num1 NUMBER := 95;
    num2 NUMBER := 85;
BEGIN
    DBMS_OUTPUT.PUT_LINE('Outer Variable num1: ' || num1);
    DBMS_OUTPUT.PUT_LINE('Outer Variable num2: ' || num2);
    DECLARE
        -- Local variables (they SHADOW the outer ones)
        num1 NUMBER := 195;
        num2 NUMBER := 185;
    BEGIN
        DBMS_OUTPUT.PUT_LINE('Inner Variable num1: ' || num1);
        DBMS_OUTPUT.PUT_LINE('Inner Variable num2: ' || num2);
    END;
END;
/

```

```

Outer Variable num1: 95
Outer Variable num2: 85
Inner Variable num1: 195
Inner Variable num2: 185

```

PL/SQL procedure successfully completed.

The inner `num1` **hides** the outer one *inside* the inner block; the outer value is untouched. To reach an outer variable that is hidden, **label** the outer block and qualify it:

```

<<outer>>
DECLARE
    x NUMBER := 1;
BEGIN
    DECLARE
        x NUMBER := 2;
    BEGIN
        DBMS_OUTPUT.PUT_LINE(x);           -- 2 (inner)
        DBMS_OUTPUT.PUT_LINE(outer.x);    -- 1 (the hidden outer one)
    END;
END outer;
/

```

## Reading input: substitution vs bind variables

`&n` (with an ampersand) is a **substitution variable of SQL Developer / SQL\*Plus, not of PL/SQL**: the *client* asks you for a value and pastes it into the text **before** the block is sent. Hence `n := &n`; . For text write `'&name'` (with quotes). A **bind variable** `:x` is a real variable shared between the client and the database ( `VARIABLE x NUMBER`; in SQL\*Plus — Chapter 13).

## 12.7 Operators and precedence

| Kind       | Operators                                                                                                                                                                                                                                                         |
|------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Arithmetic | <code>+</code> <code>-</code> <code>*</code> <code>/</code> <code>**</code> (with <code>A = 10</code> , <code>B = 5</code> : <code>A + B = 15</code> · <code>A - B = 5</code> · <code>A * B = 50</code> · <code>A / B = 2</code> · <code>A ** B = 100000</code> ) |
| Relational | <code>=</code> <code>!=</code> <code>&lt;&gt;</code> <code>~=</code> <code>^=</code> <code>&gt;</code> <code>&lt;</code> <code>&gt;=</code> <code>&lt;=</code> — return a Boolean                                                                                 |
| Comparison | <code>LIKE</code> ( <code>'Zara Ali' LIKE 'Z% A_i'</code> → TRUE) · <code>BETWEEN</code> ( <code>x BETWEEN a AND b</code> means <code>x &gt;= a AND x &lt;= b</code> ) · <code>IN</code> (set membership) · <code>IS NULL</code>                                  |
| Logical    | <code>AND</code> <code>OR</code> <code>NOT</code> (work on Booleans and produce Booleans)                                                                                                                                                                         |
| String     | <code>\ \ </code> concatenation                                                                                                                                                                                                                                   |

- With `x = 10`: `x BETWEEN 5 AND 20` → TRUE · `x BETWEEN 5 AND 10` → TRUE · `x BETWEEN 11 AND 20` → FALSE.
- With `x = 'm'`: `x IN ('a', 'b', 'c')` → FALSE · `x IN ('m', 'n', 'o')` → TRUE · `x IS NULL` → FALSE.
- With `A = TRUE`, `B = FALSE`: `A AND B` → FALSE · `A OR B` → TRUE · `NOT (A AND B)` → TRUE.
- **Comparisons involving NULL always yield NULL.**

**Operator precedence** (highest first) — for example `x := 7 + 3 * 2;` gives **13**, not 20, because `*` binds tighter than `+`:

| Order | Operator                                                                                                                                                                                                                                   | Operation                            |
|-------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------|
| 1     | <code>**</code>                                                                                                                                                                                                                            | exponentiation                       |
| 2     | <code>+</code> , <code>-</code> (unary)                                                                                                                                                                                                    | identity, negation                   |
| 3     | <code>*</code> , <code>/</code>                                                                                                                                                                                                            | multiplication, division             |
| 4     | <code>+</code> , <code>-</code> , <code>\ \ </code>                                                                                                                                                                                        | addition, subtraction, concatenation |
| 5     | <code>=</code> <code>&lt;</code> <code>&gt;</code> <code>&lt;=</code> <code>&gt;=</code> <code>&lt;&gt;</code> <code>!=</code> <code>~=</code> <code>^=</code> <code>IS NULL</code> <code>LIKE</code> <code>BETWEEN</code> <code>IN</code> | comparison                           |
| 6     | <code>NOT</code>                                                                                                                                                                                                                           | logical negation                     |
| 7     | <code>AND</code>                                                                                                                                                                                                                           | conjunction                          |
| 8     | <code>OR</code>                                                                                                                                                                                                                            | inclusion                            |

`AND` and `OR` are **short-circuited**: `IF a IS NOT NULL AND a > 5` won't evaluate `a > 5` if the first part is FALSE.

## 12.8 Conditional statements

### IF

```
IF condition THEN
    S;
END IF;
```

```

IF condition THEN
    S1;
ELSE
    S2;
END IF;

```

```

IF boolean_expression_1 THEN
    S1;          -- executes when expression 1 is true
ELSIF boolean_expression_2 THEN
    S2;          -- executes when expression 2 is true
ELSIF boolean_expression_3 THEN
    S3;
ELSE
    S4;          -- executes when none of the above is true
END IF;

```

**ELSIF is spelled with one "E"** — not `ELSEIF`, not `ELSE IF` (which would need its own `END IF;`). Ifs can be **nested**:

```

IF boolean_expression_1 THEN
    -- executes when expression 1 is true
    IF boolean_expression_2 THEN
        -- executes when expression 2 is true
        sequence_of_statements;
    END IF;
ELSE
    -- executes when expression 1 is not true
    else_statements;
END IF;

```

## CASE statement

The **simple** `CASE` compares one selector with values; the **searched** `CASE` has no selector, and each `WHEN` holds its own Boolean condition:

```

CASE selector                                -- simple CASE
    WHEN 'value1' THEN S1;
    WHEN 'value2' THEN S2;
    WHEN 'value3' THEN S3;
    ELSE Sn;                                -- default case
END CASE;

CASE                                          -- searched CASE
    WHEN selector = 'value1' THEN S1;
    WHEN selector = 'value2' THEN S2;
    ELSE Sn;
END CASE;

```

```

DECLARE
    grade CHAR(1) := 'B';
BEGIN
    CASE grade
        WHEN 'A' THEN DBMS_OUTPUT.PUT_LINE('Excellent');
        WHEN 'B' THEN DBMS_OUTPUT.PUT_LINE('Very good');
        WHEN 'C' THEN DBMS_OUTPUT.PUT_LINE('Good');
        ELSE
            DBMS_OUTPUT.PUT_LINE('No such grade');
    END CASE;
END;
/

```

Very good

**Trap — CASE statement vs CASE expression.** A CASE **expression** (in SQL, Chapter 04) with no matching branch and no ELSE returns NULL. A PL/SQL CASE **statement** with no matching branch and **no ELSE raises CASE\_NOT\_FOUND** (ORA-06592). Always write an ELSE (even ELSE NULL; ).

## A first real program — sign, then count

```

DECLARE
    n    NUMBER;
    res  VARCHAR2(100);
    t    NUMBER;
BEGIN
    n := &n;                                -- prompted at run time
    IF n > 0 THEN
        res := 'Positive';
    ELSIF n < 0 THEN
        res := 'Negative';
    ELSE
        res := 'Zero';
    END IF;

    DBMS_OUTPUT.PUT_LINE('Result : ' || res);
    DBMS_OUTPUT.PUT_LINE('Numbers : ');
    t := 1;
    LOOP
        DBMS_OUTPUT.PUT_LINE(t);
        t := t + 1;
        EXIT WHEN t > n;
    END LOOP;
END;
/

```

For input `n = 5`:

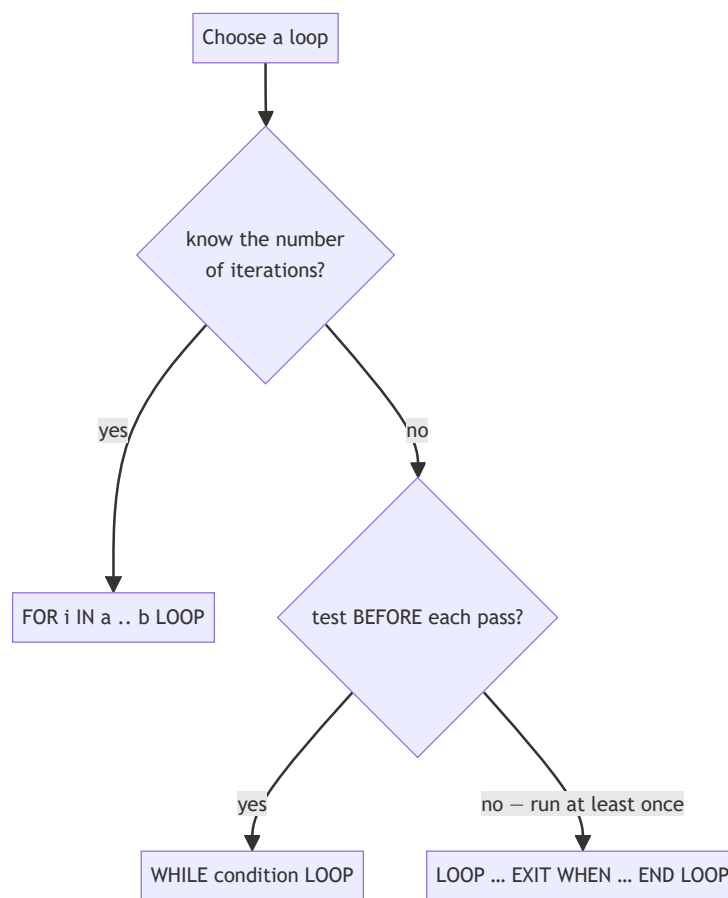
Result : Positive

Numbers :

1  
2  
3  
4  
5

(For `n = -3` it prints `Negative` , and — since a basic `LOOP` always runs **at least once** — still prints `1` before `t > n` stops it.)

## 12.9 Loops



### Basic `LOOP`

```
LOOP
    sequence_of_statements;
END LOOP;
```

The statements may be one or a block. An `EXIT` or `EXIT WHEN` is **required to break the loop** — without it the loop never ends.

```

DECLARE
    x NUMBER := 10;
BEGIN
    LOOP
        DBMS_OUTPUT.PUT_LINE(x);
        x := x + 10;
        IF x > 50 THEN
            EXIT;
        END IF;
    END LOOP;
    -- after EXIT, control resumes here
    DBMS_OUTPUT.PUT_LINE('After Exit x is: ' || x);
END;
/

```

```

10
20
30
40
50
After Exit x is: 60

```

PL/SQL procedure successfully completed.

IF cond THEN EXIT; END IF; is the same as EXIT WHEN cond; . CONTINUE (or CONTINUE WHEN cond; ) skips to the next iteration.

## WHILE loop

Repeats **while a condition is TRUE**, testing it **before each iteration** (so it may run zero times).

```

WHILE condition LOOP
    sequence_of_statements;
END LOOP;

```

```

DECLARE
    a NUMBER(2) := 10;
BEGIN
    WHILE a < 20 LOOP
        DBMS_OUTPUT.PUT_LINE('value of a: ' || a);
        a := a + 1;
    END LOOP;
END;
/

```

```
value of a: 10
value of a: 11
...
value of a: 19
```

## FOR loop

```
FOR counter IN initial_value .. final_value LOOP
    sequence_of_statements;
END LOOP;
```

Flow of control:

1. The **initial step** runs first, **once** — it declares and initialises the loop control variable.
2. The condition `initial_value .. final_value` is evaluated. If it holds, the body executes; if not, control jumps to the statement after the loop.
3. After the body, the counter is **increased (or decreased)**.
4. The condition is checked again; the cycle repeats until it fails.

Special characteristics:

- `initial_value` and `final_value` can be **literals, variables or expressions** but **must evaluate to numbers** — otherwise PL/SQL raises the predefined exception `VALUE_ERROR`.
- The initial value **need not be 1**; the **loop counter increment (or decrement) must be 1**.
- The range may be **determined dynamically at run time** (evaluated once, when the loop starts).
- The counter is **declared implicitly**, is **read-only inside the loop**, and does not exist after `END LOOP`.

```
DECLARE
    a NUMBER(2);
BEGIN
    FOR a IN 10 .. 20 LOOP
        DBMS_OUTPUT.PUT_LINE('value of a: ' || a);
    END LOOP;
END;
/
```

```
value of a: 10
value of a: 11
...
value of a: 20
```

## REVERSE

By default iteration runs from the initial value **up** to the final value. `REVERSE` iterates the other way — the counter is **decremented** after each pass — **but you must still write the bounds in ascending order**:

```

DECLARE
    a NUMBER(2);
BEGIN
    FOR a IN REVERSE 10 .. 20 LOOP          -- NOT 20 .. 10
        DBMS_OUTPUT.PUT_LINE('value of a: ' || a);
    END LOOP;
END;
/

```

```

value of a: 20
value of a: 19
value of a: 18
value of a: 17
...
value of a: 10

```

`FOR i IN 20 .. 10 LOOP` (descending, without `REVERSE`) executes **zero** times.

## Nested loops (and labels)

A loop can contain another loop of the same or a different kind:

```

LOOP
    sequence_of_statements1;
    LOOP
        sequence_of_statements2;
    END LOOP;
END LOOP;

FOR counter1 IN initial_value1 .. final_value1 LOOP
    sequence_of_statements1;
    FOR counter2 IN initial_value2 .. final_value2 LOOP
        sequence_of_statements2;
    END LOOP;
END LOOP;

WHILE condition1 LOOP
    sequence_of_statements1;
    WHILE condition2 LOOP
        sequence_of_statements2;
    END LOOP;
END LOOP;

```

**Label** a loop with `<<name>>` so that `EXIT` / `CONTINUE` can target the *outer* one:

```

BEGIN
  <<outer_loop>>
  FOR i IN 1 .. 3 LOOP
    FOR j IN 1 .. 3 LOOP
      EXIT outer_loop WHEN i * j = 4;      -- leaves BOTH loops
      DBMS_OUTPUT.PUT_LINE(i || ' x ' || j || ' = ' || (i * j));
    END LOOP;
  END LOOP outer_loop;
END;
/

```

```

1 x 1 = 1
1 x 2 = 2
1 x 3 = 3
2 x 1 = 2

```

## Practice program — factorial, even numbers, reverse

```

DECLARE
  n NUMBER;
  f NUMBER := 1;
  t NUMBER;
BEGIN
  n := &n;
  t := n;
  WHILE t > 0 LOOP
    f := f * t;
    t := t - 1;
  END LOOP;
  DBMS_OUTPUT.PUT_LINE('Factorial : ' || f);

  DBMS_OUTPUT.PUT_LINE('Even Numbers : ');
  FOR i IN 1 .. n LOOP
    IF MOD(i, 2) = 0 THEN
      DBMS_OUTPUT.PUT_LINE(i);
    END IF;
  END LOOP;

  DBMS_OUTPUT.PUT_LINE('Reverse : ');
  FOR i IN REVERSE 1 .. n LOOP
    DBMS_OUTPUT.PUT_LINE(i);
  END LOOP;
END;
/

```

For `n = 5`:

```
Factorial : 120
Even Numbers :
2
4
Reverse :
5
4
3
2
1
```

## A recursive relative — factorial as a function

$n! = n \times (n-1)!$  — see recursion in Chapter 13.

## 12.10 Strings

PL/SQL has **three kinds of strings**:

- **Fixed-length strings** — you specify the length when declaring; the value is **right-padded with spaces** to that length.
- **Variable-length strings** — you specify a **maximum** (up to 32,767); **no padding**.
- **Character large objects (CLOBs)** — variable-length strings up to **128 terabytes**.

A **string literal** is enclosed in single quotes: `'This is a string literal.'` or `'hello world'`. To include a **single quote** inside a literal, **type two single quotes** next to each other: `'this isn't what it looks like'`.

Oracle's string types are `CHAR`, `NCHAR`, `VARCHAR2`, `NVARCHAR2`, `CLOB` and `NCLOB`. Those **prefixed with N** use the **national character set** and store Unicode. A variable-length string **needs a maximum length**.

```
DECLARE
    name          VARCHAR2(20);
    company       VARCHAR2(30);
    introduction   CLOB;
    choice        CHAR(1);
BEGIN
    name          := 'John Smith';
    company       := 'Infotech';
    introduction  := ' Hello! I''m John Smith from Infotech.';
    choice        := 'y';
    IF choice = 'y' THEN
        DBMS_OUTPUT.PUT_LINE(name);
        DBMS_OUTPUT.PUT_LINE(company);
        DBMS_OUTPUT.PUT_LINE(introduction);
    END IF;
END;
/
```

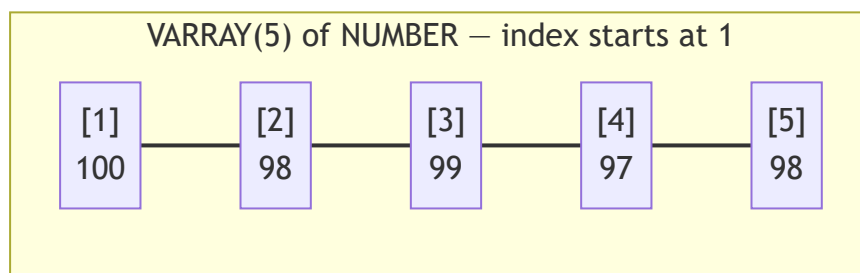
```
John Smith
Infotech
Hello! I'm John Smith from Infotech.

PL/SQL procedure successfully completed.
```

**String functions in PL/SQL** are the same as in SQL (Chapter 09): `ASCII`, `CHR`, `CONCAT`, `INITCAP`, `INSTR` (with optional `start` and `occurrence`), `INSTRB` (bytes), `LENGTH`, `LENGTHB`, `LOWER`, `LPAD`, `LTRIM`, `NANVL`, `NVL`, `NVL2`, `REPLACE`, `RPAD`, `RTRIM`, `SOUNDEX`, `SUBSTR`, `SUBSTRB`, `TRIM`, `UPPER`. Two details worth knowing: `LPAD(x, width [, pad_string])` pads with **spaces** when `pad_string` is omitted, and `NVL2(x, value1, value2)` returns `value1` if `x` is **not** `NULL`, otherwise `value2`.

## 12.11 Arrays — the `VARRAY`

PL/SQL's array-like structure is the `VARRAY` ("variable-size array"). It stores a **fixed-size sequential collection of elements of the same type**. A varray is an **ordered collection**, best thought of as a collection of variables of one type; all varrays consist of **contiguous memory locations** — the lowest address is the first element, the highest the last.



- Each element has an **index** (starting at **1**).
- It has a **maximum size**, which can be changed dynamically.
- It is part of the **collection** family (nested tables and associative arrays are the others).

### Creating a varray type

**At schema level** — stored in the database with `CREATE TYPE`; you must give the **maximum size** and the **element type**:

```
CREATE OR REPLACE TYPE varray_type_name IS VARRAY(n) OF <element_type>;
```

`n` is the **maximum number of elements**, and `<element_type>` the element's data type. The maximum size can be changed later with `ALTER TYPE`.

```
CREATE OR REPLACE TYPE namearray AS VARRAY(3) OF VARCHAR2(10);
/
```

```
Type created.
```

**Inside a PL/SQL block** (local to the block):

```

TYPE varray_type_name IS VARRAY(n) OF <element_type>;

TYPE namearray IS VARRAY(5) OF VARCHAR2(10);
TYPE grades IS VARRAY(5) OF INTEGER;

```

## Example 1 — names and marks

```

DECLARE
    TYPE namesarray IS VARRAY(5) OF VARCHAR2(10);
    TYPE grades IS VARRAY(5) OF INTEGER;
    names namesarray;
    marks grades;
    total INTEGER;
BEGIN
    names := namesarray('Kavita', 'Pritam', 'Ayan', 'Rishav', 'Aziz');
    marks := grades(98, 97, 78, 87, 92);
    total := names.COUNT;
    DBMS_OUTPUT.PUT_LINE('Total ' || total || ' Students');
    FOR i IN 1 .. total LOOP
        DBMS_OUTPUT.PUT_LINE('Student: ' || names(i) || ' Marks: ' || marks(i));
    END LOOP;
END;
/

```

```

Total 5 Students
Student: Kavita Marks: 98
Student: Pritam Marks: 97
Student: Ayan Marks: 78
Student: Rishav Marks: 87
Student: Aziz Marks: 92

PL/SQL procedure successfully completed.

```

## Example 2 — the lab version

```

DECLARE
    TYPE num_grades IS VARRAY(5) OF NUMBER;
    my_num num_grades;
BEGIN
    my_num := num_grades(100, 98, 99, 97, 98);
    FOR i IN 1 .. 5 LOOP
        DBMS_OUTPUT.PUT_LINE(my_num(i));
    END LOOP;
END;
/
-- Note: the index starts from 1

```

100  
98  
99  
97  
98

## Useful varray facts

|                       |                                                                                                                                                                                                                          |
|-----------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Initialise before use | <code>my_num := num_grades(...)</code> — the <b>constructor</b> — otherwise <code>ORA-06531: Reference to uninitialized collection</code>                                                                                |
| Methods               | <code>COUNT</code> (elements now), <code>LIMIT</code> (maximum size), <code>EXTEND</code> (add an empty slot), <code>FIRST</code> , <code>LAST</code> , <code>EXISTS(i)</code> , <code>TRIM</code> , <code>DELETE</code> |
| Bounds                | bigger than <code>LIMIT</code> → <code>ORA-06532: Subscript outside of limit</code> ; beyond <code>COUNT</code> → <code>ORA-06533: Subscript beyond count</code>                                                         |
| Index base            | <b>1</b> (not 0)                                                                                                                                                                                                         |
| Dense                 | a varray has no gaps (unlike an associative array)                                                                                                                                                                       |

## Interview corner

**Q. What is PL/SQL? Why use it over plain SQL?** Oracle's procedural extension of SQL: blocks with variables, control flow, exceptions and reusable stored code — fewer round trips, tighter integration, business logic in the database.

**Q. Sections of a PL/SQL block?** `DECLARE` (optional) · `BEGIN` (mandatory) · `EXCEPTION` (optional) · `END` ; .

**Q. Anonymous block vs stored procedure?** Anonymous: unnamed, not stored, compiled every run. Stored: named, saved in the database, compiled once, reusable with parameters.

**Q. What does %TYPE do?** Declares a variable with the same data type as a table column (or another variable), so it adapts if the column changes.

**Q. := vs = ?** `:=` assigns; `=` compares.

**Q. Difference between NOT NULL and CONSTANT in a declaration?** `NOT NULL` forbids NULL (needs an initial value); `CONSTANT` forbids *changing* the value (needs an initial value).

**Q. What happens if SELECT ... INTO returns no rows or several?** `NO_DATA_FOUND` / `TOO_MANY_ROWS` .

**Q. LOOP vs WHILE vs FOR ?** `LOOP` — runs at least once, needs `EXIT` ; `WHILE` — condition tested first (may run 0 times); `FOR` — a known number of iterations, counter implicit and read-only.

**Q. How do you loop backwards from 10 to 1?** `FOR i IN REVERSE 1 .. 10 LOOP` — bounds stay ascending.

**Q. CASE statement without ELSE — what happens if nothing matches?** It raises `CASE_NOT_FOUND` ( `ORA-06592` ).

**Q. What is the scope of a variable declared in an inner block?** Only that block; outer-block variables are visible in inner blocks (and can be shadowed).

**Q. Why does IF x = NULL never work?** NULL compared with anything is UNKNOWN; use `IS NULL` .

**Q. How do you print output? Why do I see nothing?** `DBMS_OUTPUT.PUT_LINE` ; run `SET SERVEROUTPUT ON` ; first.

## Quick recap

---

- **PL/SQL = SQL + procedural code** in **blocks**: `DECLARE` (opt) · `BEGIN` (must) · `EXCEPTION` (opt) · `END;` + `/`.
- **Anonymous** blocks are unnamed and not stored; **named** blocks (procedures, functions, packages, triggers) are stored; a **local** subprogram lives only inside its enclosing block.
- Types: numeric ( `PLS_INTEGER` , `NUMBER` , ...), character ( `CHAR` , `VARCHAR2` , `NCHAR` ...), Boolean (PL/SQL only), datetime, LOBs ( `BFILE` , `BLOB` , `CLOB` , `NCLOB` ); **subtypes** rename/limit a base type.
- Declare with `name [CONSTANT] type [NOT NULL] [:= value];` %TYPE follows a column; `SELECT ... INTO` needs **exactly one row**; inner variables **shadow** outer ones.
- `NULL` ≠ anything, even itself — test with `IS NULL` ; an `IF` on UNKNOWN takes the `ELSE` .
- Precedence: `**` → unary → `*` / `/` → `+` - `||` → comparison → `NOT` → `AND` → `OR` .
- Control: `IF/ELSIF/ELSE` , `CASE` (statement needs an `ELSE` ), `LOOP ... EXIT WHEN` , `WHILE` , `FOR [REVERSE] a..b` .
- Strings: fixed / variable / CLOB; `'` inside a literal is a quote. `VARRAY` : bounded, 1-based, must be initialised with its constructor.